

Data Representation

Number systems

Zain Merchant

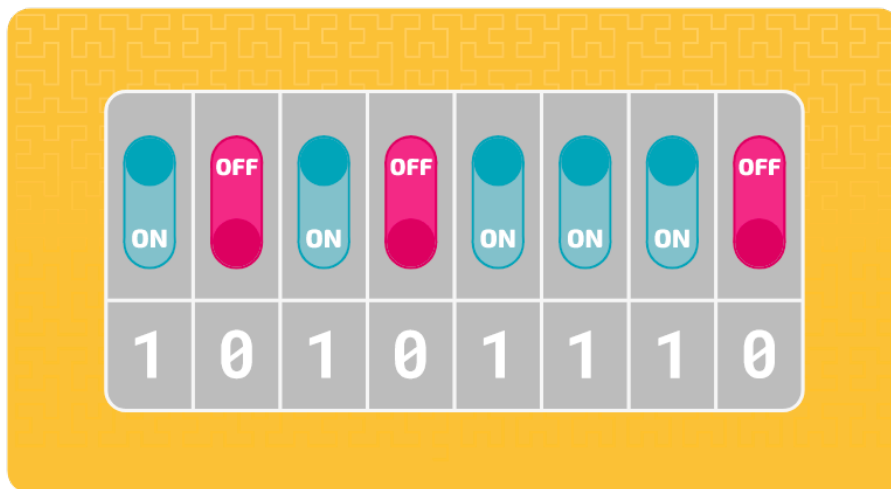
Representation of numbers

Numbers in everyday life are usually represented using the digits 0 to 9, but this is not the only way in which a number can be represented. There are multiple number base systems, which determine which digits are used to represent a number. The number system that we are most familiar with is called **denary** or **decimal (base-10)**, but **binary (base-2)** and **hexadecimal (hex or base-16)** are also used by computers. You can perform arithmetic calculations on numbers written in other base notations, and even convert numbers between bases.

Binary and denary

Many ancient cultures developed the counting system that we use today, known as the **decimal system**. It allows us to use ten values and it is likely that this common approach was developed because of the fact humans have ten fingers/digits to count with. You may have also heard this system referred to as **denary** or **base-10**.

Computers obviously don't have fingers, but instead use tiny switches called transistors that allow electricity to be **on** or **off** in a circuit. These circuits are combined to represent data and the two states of **on** or **off** are represented as **1** or **0**. This is known as the **binary** or **base-2** as only two values can be used. Combinations of **1s** and **0s** can be used by a computer to represent any type of information (e.g. numbers, text, images, sound, program instructions).



Base - 10 (Denary)

The denary system is a method of assigning a **place value** to numbers.

A place value is the numerical value of a digit that appears within a number. For example, take the number: 1892_{10}

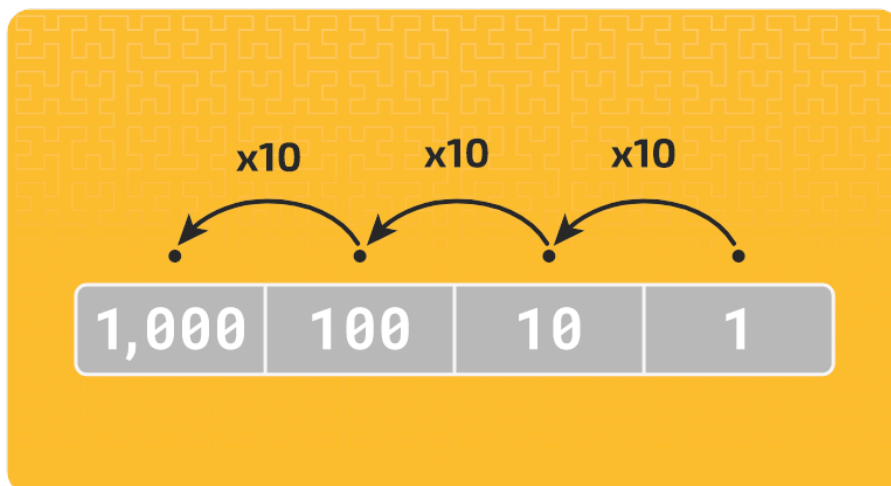
The place value of 1892_{10} is one thousand.

1000	100	10	1
1	8	9	2

In this number, there are:

- 1 thousand
- 8 hundreds
- 9 tens
- 2 ones

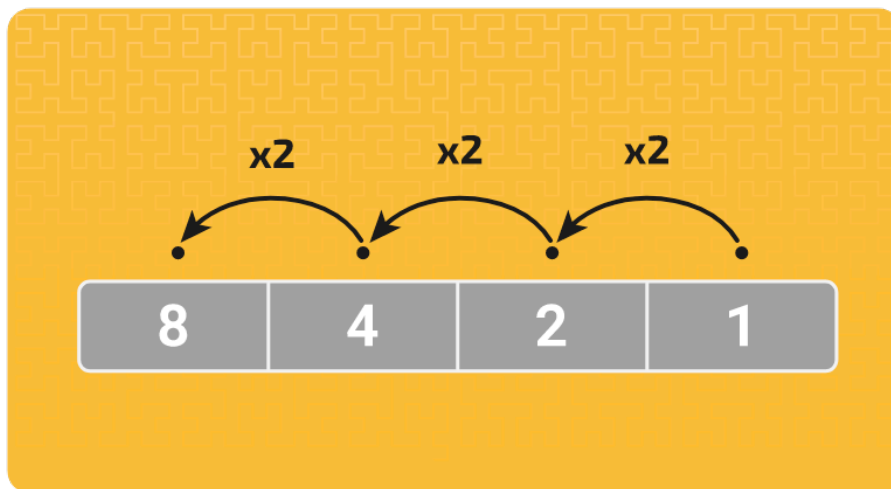
To work out what the place values are, you start from the first column on the right where the **1** place value is and multiply by 10 as you move from right to left.



Base - 2 (Binary)

Binary is a base-2 number system. It only uses the digits 0 and 1. To understand how a binary value translates to a denary, you need to understand the **place values** for a base-2 system.

To work out what the place values are, you use the same process as you did with the denary system and start from the first column on the right where the **1** place value is. But when using **base-2**, you multiply by 2 each time as you move from right to left.



Take the following binary number: 0100₂

The place value of 1 is 4

8	4	2	1
0	1	0	0

In this number, there are:

- 0 eights
- 1 four
- 0 twos
- 0 ones

Converting from binary to denary

To convert from **binary** to **denary**, you need to know the place value of each digit in the number.

When you are just starting to learn how to do this, it is a good idea to always use a **table** to help you with the conversion.

For example, if working with a **4-bit** binary number, use the following table:

8	4	2	1

For an **8-bit** binary number, use the following table:

128	64	32	16	8	4	2	1

Example 1

Take the **4-bit** binary number 1011_2 and place the digits from right to left in the table:

8	4	2	1
1	0	1	1

By doing this, you can see the value of each **bit**. Look at the **place value** for each bit represented as a

$$8+2+1=11$$

Therefore, the binary number 1011_2 is 11_{10} in denary.

Example 2

A **byte** is equal to **8 bits**. To convert the following byte, follow the same process but this time use the table with eight columns.

Convert the following binary value into denary: 10001101_2

128	64	32	16	8	4	2	1
1	0	0	0	1	1	0	1

$$128 + 8 + 4 + 1 = 141_{10}$$

Therefore, the binary value of 10001101_2 is equal to the denary value of 141_{10}

Converting from denary to binary

Converting from denary to binary is a very similar to the process you used to convert binary to denary.

The main two differences are as follows:

- You now need to work from **left to right**
- You need to **subtract**, not add

Again, it is a good idea to always use a **table** like this one.

128	64	32	16	8	4	2	1

Example 1

To describe the process, the denary number 5 will be converted into its binary equivalent.

Using your table, you start by looking for the first place value that is less than 5 and place a 1 in that column. In this case, it is the column with the place value of 4 as all the place values to the left of this (8, 16, 32, etc.) are all greater than the value of 5.

128	64	32	16	8	4	2	1
					1		

Now that you have placed the 1 in that column, fill the empty spaces to the **left** with zeros:

128	64	32	16	8	4	2	1
0	0	0	0	0	1		

Next, take the place value away from your current number (5).

$$5 - 4 = 1$$

The next step is to look for the highest remaining place value for the remaining number. As in this instance the remaining number is 1, this neatly fits into the 1 column. Fill in the gaps with 0s.

128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	1

Now you have your conversion:

5_{10} in denary is 101_2 in binary.

Example 2

In this example, follow the same process, but this time, convert the larger denary value of 76₁₀

Step 1

Look for the highest value that fits into the number 76 and place a 1 in that column. In this case, it is the column with the place value of 64 as the place value to the right of this (128) is greater than 76.

128	64	32	16	8	4	2	1
0	1						

Now that you have placed the 1 in that column, you then take the value of the place value away from your current number (76).

$$76 - 64 = 12$$

Step 2

Repeat the same process again and find the highest value that fits into the number you have remaining (12). In this case, it is the column with the place value 8.

128	64	32	16	8	4	2	1
0	1	0	0	1			

Complete the same calculation as last time to see what you have remaining:

$$12 - 8 = 4$$

Step 3

The remainder is now 4 and there is a place value for 4. Place a 1 in that column.

128	64	32	16	8	4	2	1
0	1	0	0	1	1		

$$4 - 4 = 0$$

Now that there is no remainder, fill in the remaining columns with zeros.

128	64	32	16	8	4	2	1
0	1	0	0	1	1	0	0

Answer: The denary value of 76_{10} is equal to the binary value 1001100_2

Base - 16 (Hexadecimal)

Hexadecimal (base-16, hex) is often used in computer science. This system uses a base of **16 digits**, i.e. 16 unique symbols are combined to make up all other numbers. There are only ten symbols in the denary number system (0–9), and so in hexadecimal, a further six symbols (the characters A–F) are used to represent the remaining six digits.

The 16 digits that form the base of the hexadecimal system correspond to the denary values 0–15. Also, each hex digit is equivalent to four binary digits. Here are the sixteen digits that form the base of the hex system:

Denary	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Hexadecimal is used to represent a binary value. For example, look at how the denary number 16_{10} is represented as a binary number and in hexadecimal:

Binary	10100001
Hexadecimal	A1

Why is hexadecimal used as shorthand for binary?

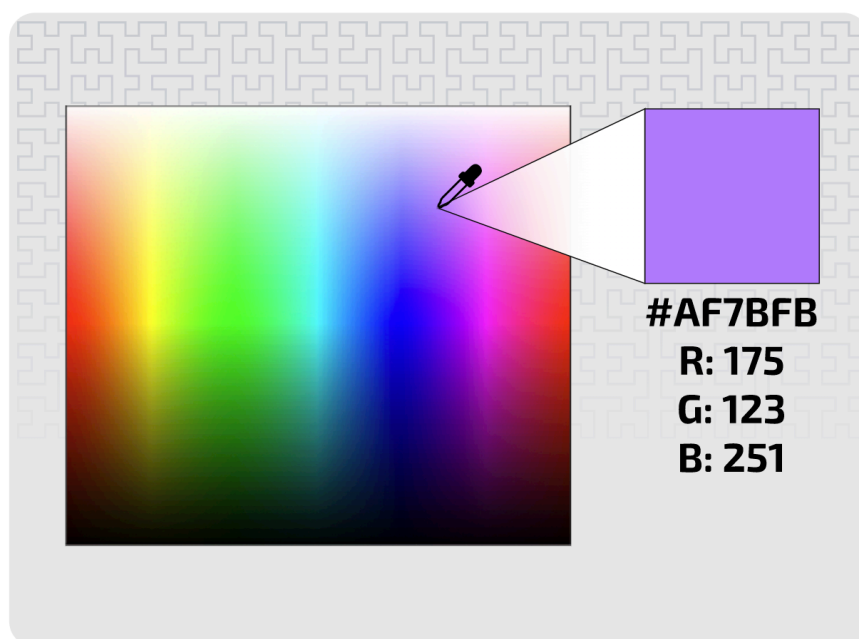
Hexadecimal is often used by people instead of binary because:

- It is easier to read and interpret
- It uses fewer digits to represent the same value
- Compared to binary, it is less likely that a digit will be written down incorrectly

Below are some areas of computing where you might come across the use of hexadecimal.

Programming with colours

Often, when you pick a colour in a program, a hexadecimal value is assigned to that colour.



Many programming languages and software applications allow programmers, designers, and digital artists to enter in their choice of colour as a hexadecimal. This is because, compared to binary, the values are much easier to remember and to write out when they need to be used.

Most electronic screens use **RGB** to display colour. Each colour combines 8 bits for a shade of red (R), 8 bits for a shade of green (G), and 8 bits for a shade of blue (B). Therefore, to represent any RGB colour, **24** bits are needed.

It's very hard for anyone to remember a combination of 24 bits; it's much easier to remember a hexadecimal value of just **6** digits.

For example, the colour orange can be depicted as $FFA500_{16}$ instead of 1111111010010100000000.

Media Access Control (MAC addresses)

A media access control (MAC) address is a number that relates to a network interface controller. MAC addresses are usually displayed as a set of hexadecimal digits separated by colons.

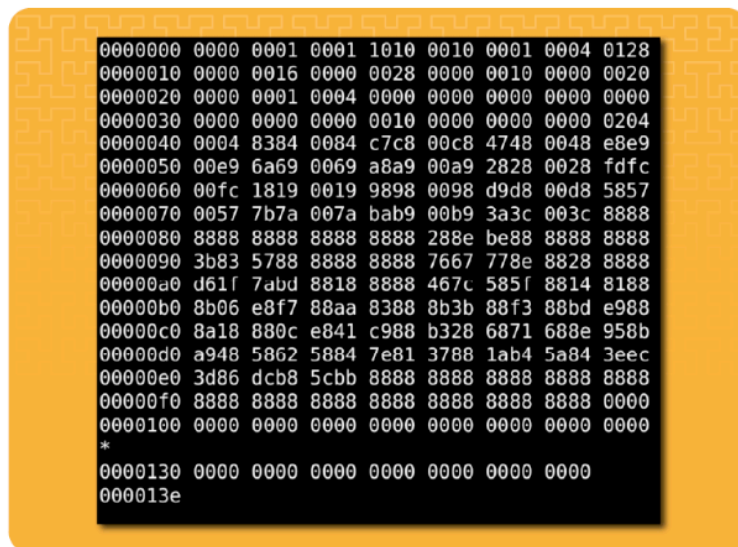
Result for: **B8:27:EB:10:20:AB**

Address Prefix	B8:27:EB
Vendor / Company	Raspberry Pi Foundation
Start Address	B827EB000000
End Address	B827EBFFFFFF

Memory dumps

A memory dump typically appears on a screen when the computer has crashed. It's called a memory dump as it is outputting the current state of the computer's working memory to help the user in debugging the error.

By representing the memory dump as hexadecimal instead of binary numbers, the length of the memory dump when it is displayed is reduced by 75%.



Converting binary to hexadecimal

To convert between binary and hexadecimal, you will use a process that involves:

1. Splitting your binary number into nibbles (sets of 4 bits)
2. Calculate the value of each nibble in denary using the place values for the 4-bit numbers
3. Convert the denary value into the corresponding hexadecimal digit
4. Read the hexadecimal number from left to right

Reminder:

0 to 9 in denary is the same in hexadecimal. For example $8_{10} = 8_{16}$

The denary numbers 10_{10} to 15_{10} are represented as follows:

Hex	A	B	C	D	E	F
Denary	10	11	12	13	14	15

For example: $12_{10} = C_{16}$

This is how to convert the following binary number into hexadecimal: 10011011_2

Step 1

Split the number into two nibbles:

1001 and 1011

Step 2

To calculate the value of each nibble in denary, use the place holder table below to help you:

8	4	2	1	8	4	2	1
1	0	0	1	1	0	1	1

- Nibble 1 (1001): $8 + 1 = 9$
- Nibble 2 (1011): $8 + 2 + 1 = 11$

Step 3

Convert the denary value into the corresponding hexadecimal digit:

- Nibble 1 (1001): $9_{10} = 9_{16}$
- Nibble 2 (1011): $11_{10} = B_{16}$

Step 4

Read the hexadecimal number from left to right:

$$10011011_2 = 9B_{16}$$

Converting hexadecimal to binary

To convert a hexadecimal number into binary:

1. Take each hex digit separately and find its equivalent denary value
2. Convert each denary value to a **nibble** (4-bit binary number) using appropriate place values for each of the digits; each value **has to be expressed using four digits**
3. Combine the nibbles and read the binary number from left to right

Example:

This is how to convert the hexadecimal number $6B16$.

Step 1

Find the equivalent denary number for each of the hex digits:

- $616 = 610$
- $F16 = 1110$

Step 2

Convert each denary number into a **4-bit binary number**; the place values for each set of binary four digits are:

8	4	2	1	8	4	2	1
0	1	1	0	1	0	1	1
$4 + 2 = 6$				$8 + 2 + 1 = 11$			

Step 3

Combine the nibbles and read the binary number from left to right:

$$6B_{16} = 01101011_2$$

Step 4

If you were required to convert $6B_{16}$ to a denary number, now that you have the binary number, you can use the same method as converting binary to denary as follows:

128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1

$$64 + 32 + 8 + 2 + 1 = 107$$

Therefore you can say that:

$$6B_{16} = 107_{10}$$

Binary Addition

Rule 1: $0 + 0 = 0$

Rule 2: $0 + 1 = 1$ or $1 + 0 = 1$

Rule 3: $1 + 1 = 0$ carry 1

Rule 4: $1 + 1 + 1 = 1$ carry 1

When performing an addition, you may be given two or more binary numbers to add together. Put the numbers above each other, with the binary numbers aligned to the right, then look at each column from the right, one at a time. If there are 8 bits, look at the column with the 8th bit in and find which rule applies to it. Then move to the 7th. Carried digits are put in the column to the left, and they count when applying the rules.

Worked example

Add the binary numbers 01111011 and 01101000.

Step 1: put the numbers together (these are in a table to help to get you started).

	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0

Step 2: look at the rightmost column, $1 + 0$.

Which rule does this follow? Rule 2. So the answer is 1.

	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
								1

Step 3: look at the next rightmost column, $1 + 0$.

Rule 2 again. Fill in the answer.

	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
							1	1

Step 4: next column, $0 + 0$.

Rule 1. The answer is 0.

	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
						0	1	1

Step 5: next column, $1 + 1$.

Rule 3. The answer is 0, carry 1. The carry goes below the column to the left.

				1				
	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
					0	0	1	1

Step 6: next column (there is now a bit in the carry that needs to be taken into account). $1 + 0 + 1$.

Ignore the 0, there are two 1s, so this follows Rule 3. The answer is 0 carry 1.

				1	1			
	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
				0	0	0	1	1

Step 7: next column (including the carry), $1 + 1 + 1$.

There are three 1s, so this follows Rule 4. The answer is 1 carry 1.

				1	1	1		
	0	1	1	1	1	0	1	1
+	0	1	1	0	1	0	0	0
			1	0	0	0	1	1

Step 8: next column (including the carry), $1 + 1 + 1$.

Rule 4. 1 carry 1.

	1		1		1		1	
	0		1		1		1	
+	0		1		0		1	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	

Step 9: next column (including the carry), $0 + 0 + 1$.

There is one 1, so this follows Rule 2. The answer is 1.

	1		1		1		1	
	0		1		1		1	
+	0		1		0		1	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	

Once you have completed an addition, convert the binary numbers to check you have done it correctly.

	1		1		1		1	
	0		1		1		1	
+	0		1		0		1	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	
			1		0		0	

= 123

= 104

= 227

Worked example

Add the binary numbers 10110110 and 11000111.

Step 1: put the numbers together and complete the rightmost column (two 1s, Rule 3).

	1	0	1	1	0	1	1	1
+	1	1	0	0	0	1	1	1

Step 2: check the second column (three 1s, Rule 4).

And continue until the end.

	1				1	1	1	
	1	0	1	1	0	1	1	1
+	1	1	0	0	0	1	1	1
	0	1	1	1	1	1	1	0

There is an extra carry left over on this one. This is called **overflow**. It means that the two (in this case) 8-bit numbers added together need more than 8 bits. They need 9. Show this in the examination to make it clear you know what has happened. You may also be asked what it is and why it is there.

Logical Shifts

Left shift

A logical left shift shifts all of the bits in a binary string to the left by a specified number of places.

For example, a left shift by one place would involve:

- Moving all of the bits in the string one place to the left
- Discarding the most significant (leftmost) bit
- Putting a 0 into the empty place on the right

If the string represents a number, this operation is equivalent to multiplying the number by 2. Each shift to the left will multiply the number by 2, so performing a shift three places to the left on a binary number is the same as multiplying the number by

$2^3 = 8$.

Consider this example of multiplying 14 by 8. The binary value is shifted left three times to obtain the result 112:

	128	64	32	16	8	4	2	1
14	0	0	0	0	1	1	1	0
28	0	0	0	1	1	1	0	0
56	0	0	1	1	1	0	0	0
112	0	1	1	1	0	0	0	0

Right shift

A logical right shift shifts all of the bits in a binary string to the right by a specified number of places.

For example, a right shift by one place would involve:

- Moving all of the bits in the string one place to the right
- Discarding the least significant (rightmost) bit
- Putting a 0 into the empty place on the left

If the string represents a number, this operation is equivalent to dividing the number by 2. In general terms, you can say that shifting a binary number right by n places has the effect of dividing the number by 2^n .

Signed integers in binary

Whole numbers such as 7, 12 and 3988 are called **integers**. **Unsigned** integers have positive values by definition, while **signed** integers can be **positive** or **negative**; the numbers that are larger than zero are called positive, and the ones smaller than zero are called negative. In denary, negative integers are represented using a minus symbol before the value of the number, e.g. -19 . In binary, there are several ways to represent signed integers, the most common being **two's complement**.

Denary to two's complement

If the denary number is positive, then conversion is the same as to binary. However, because it is positive the first bit (the most significant bit) must be a 0 to show it is positive.

For example, 23 in binary is 10111. However, this starts with a 1, which would indicate that the number is negative. Add a 0 at the front to indicate that it is positive: 010111.

If the denary number is negative, then convert it to two's complement form. There are a number of ways of doing this, but we'll stick with one method here. You convert the denary number to binary (as normal), then flip every bit (if it's a 1, replace it with a 0, if it's a 0 replace it with a 1), then add binary 1 to it.

Worked example

Convert the denary number -35 into two's complement.

Step 1: write $+35$ in binary (add 0 at the left to show it is positive).

00100011

Step 2: is the number you are converting negative? Yes, so flip every bit.

11011100

Step 3: add 1.

	1	1	0	1	1	1	0	0
+	0	0	0	0	0	0	0	1
	1	1	0	1	1	1	0	1

Step 4: write the answer.

11011101

Data Representation

Text, sound and images

Zain Merchant

ASCII and Unicode

To represent text digitally, each character needs to have its own **unique bit-pattern**. Bit-patterns are combinations of 1s and 0s used to represent data inside of a computer. The bit-pattern used for each character becomes a numeric **character code**.

A character can be any of the following:

- Letters (upper and lower case letters have separate codes)
- Punctuation (e.g. ?/\£\$)
- Numbers (0–9)
- Non-printing commands (e.g. Enter, Delete, F1)

For computers to be able to communicate and exchange text between each other efficiently, they must have an agreed standard that defines which character code is used for which character. A standardised collection of characters and the bit-patterns used to represent them is called a **character set**.

ASCII

ASCII stands for 'American Standard Code for Information Interchange'. It was defined in 1963 and was one of the most common character sets used. It started by using **7 bits** to represent characters, which allowed for a maximum of 128 (2⁷) characters to be represented.

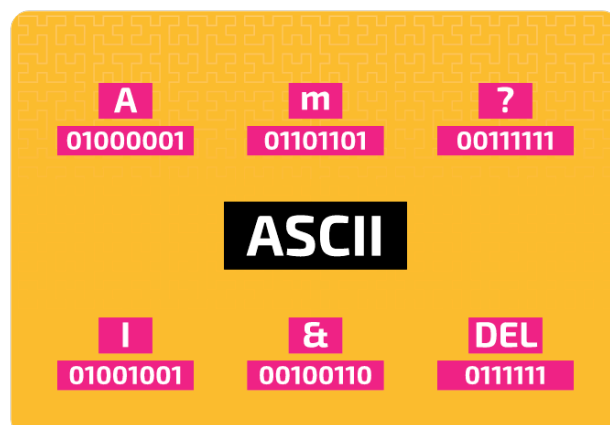
These days, **8 bits** (1 byte) are used to store each character in the ASCII character set. The original coding system remains, but each code now has a preceding 0, so there are still 128 bit-patterns in the set. The eighth bit was sometimes used as a parity bit for checking for errors during the transmission of data.

When text is encoded and stored using ASCII, each of the characters is assigned a denary (decimal) character code, which is represented and stored in the computer as binary.

If you look carefully at the ASCII representation of each character, you might notice some patterns. For example:

Character	Character code in denary	Character code in binary
a	97	0110 0001
b	98	0110 0010
c	99	0110 0011

As you can see, a is represented by 97, b is represented by 98, and c is represented by 99. This means that if you know the denary value of a character, you can easily work out the denary values of the previous and subsequent characters.



Extended ASCII

There are also extensions on this standard, such as **extended ASCII**, which uses 8 bits to represent characters, which raises the possible range of characters to $2^8 = 256$.

Unicode

The problem with ASCII is that it only allows you to represent a small number of characters (128 for standard ASCII). This might be enough to represent the characters in the English alphabet, but it is not sufficient to represent all of the languages and scripts in the world, and all of the possible numbers and symbols. For example, ASCII can't possibly store the hundreds of thousands of characters in the below scripts in just 8 bits.

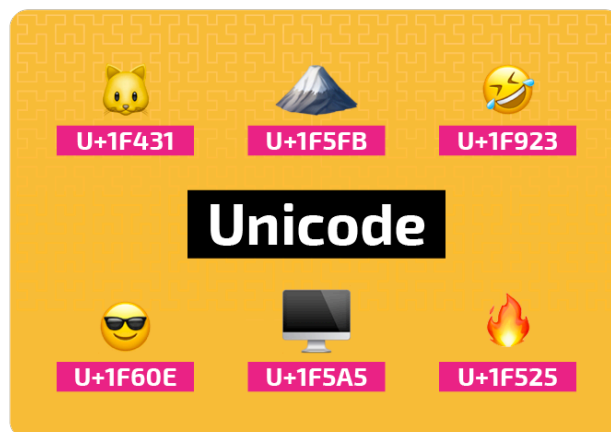
- Chinese characters 汉字
- Japanese characters 漢字
- Cyrillic Кириллица
- Gujarati ગુજરાતી
- Urdu اردو
- Greek ελληνικά

Moreover, the widespread use of the World Wide Web made it more important to have a universal international coding system, as the range of platforms and programs has increased dramatically, with more developers from around the world using a much wider range of characters.

The character set that is most commonly used instead is Unicode. Each Unicode character can be encoded on a computer with three different encoding standards, which differ based on the minimum number of bits used:

Goals of unicode

- create a universal standard that covered all languages and all writing systems
- produce a more efficient coding system than ASCII
- adopt uniform encoding where each character is encoded as 16-bit or 32-bit code
- create unambiguous encoding where each 16-bit and 32-bit value always represents the same character
- reserve part of the code for private use to enable a user to assign codes for their own characters and symbols (useful for Chinese and Japanese character sets, for example).



Character codes for numeric digits

A number can be represented as a set of characters. For example, the number 35 can be represented as the characters '3' and '5'. When a denary digit (from 0 to 9) is processed as a character, the computer uses the binary pattern of its **character code**, instead of the binary representation of that digit. For example, the binary representation of the number 35 using 8 bits is 00100011_2 , but the binary pattern for '35' is 0011001100110101_2 . This is because the character code for '3' using 8-bit ASCII is $51_{10} = 00110011_2$ and the character code for '5' is $53_{10} = 00110101_2$. Therefore, it is important that you can tell the difference between the binary representation of a denary number, and the (different) binary pattern for that number when it is stored as a set of characters.

Representation of images

All data on a computer system is represented using binary patterns, which are sequences of 1s and 0s. In order to represent an image, one method is to store it as if it were a grid of coloured squares, with each colour represented by a unique binary pattern. The image dimensions and the number of colours used are factors that affect the size of the image file.

At a more advanced level, you will learn that images can also be stored as mathematical equations describing shapes, which are then rendered back into an image when viewed by the user. It is useful to know the benefits and drawbacks of each image representation method in order to decide the correct format in which to save a particular image.

Bitmap images

Bitmap images are made up of pixels (picture elements); an image is made up of a two-dimensional matrix of pixels. Pixels can take different shapes such as:



Each pixel can be represented as a binary number, and so a bitmap image is stored in a computer as a series of binary numbers, so that:

- a black and white image only requires 1 bit per pixel – this means that each pixel can be one of two colours, corresponding to either 1 or 0
- if each pixel is represented by 2 bits, then each pixel can be one of four colours ($2^2 = 4$), corresponding to 00, 01, 10, or 11
- if each pixel is represented by 3 bits then each pixel can be one of eight colours ($2^3 = 8$), corresponding to 000, 001, 010, 011, 100, 101, 110, 111.

The number of bits used to represent each colour is called the colour depth. An 8 bit colour depth means that each pixel can be one of $2^8 = 256$ colours (because $2^8 = 256$). Modern computers have a 24 bit colour depth, which means over 16 million different colours can be represented. With x pixels, 2^x colours can be represented as a generalisation. Increasing colour depth also increases the size of the file when storing an image.

Image resolution refers to the number of pixels that make up an image; for example, an image could contain 4096×3072 pixels (12 582 912 pixels in total).

The resolution can be varied on many cameras before taking, for example, a digital photograph. Photographs with a lower resolution have less detail than those with a higher resolution.

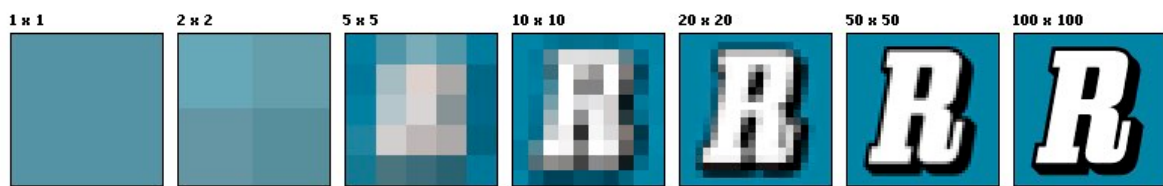


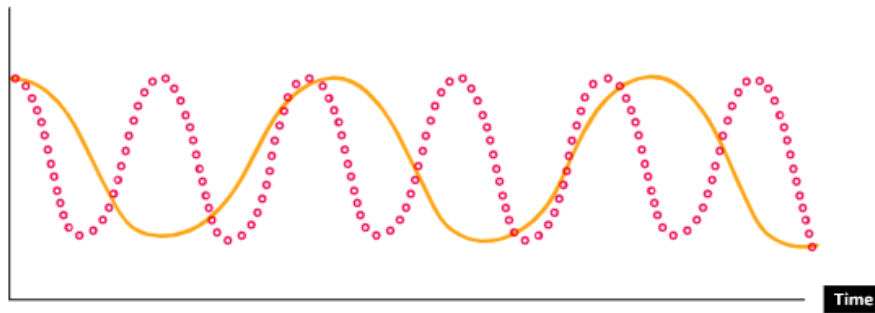
Image 'A' has the highest resolution and 'E' has the lowest resolution. 'E' has become pixelated ('fuzzy'). This is because there are fewer pixels in 'E' to represent the image.

The main drawback of using high resolution images is the increase in file size. As the number of pixels used to represent the image is increased, the size of the file will also increase. This impacts on how many images can be stored on, for example, a hard drive. It also impacts on the time to download an image from the internet or the time to transfer images from device to device. A certain amount of reduction in resolution of an image is possible before the loss of quality becomes noticeable.

Representation of sound

Soundwaves are vibrations in the air. The human ear senses these vibrations and interprets them as sound.

Each sound wave has a frequency, wavelength and amplitude. The amplitude specifies the loudness of the sound.

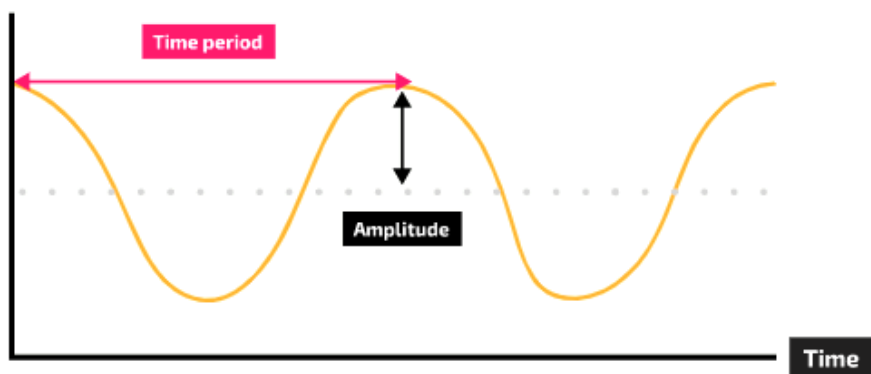


Sound waves vary continuously. This means that sound is analogue. Computers cannot work with analogue data, so sound waves need to be sampled in order to be stored in a computer. Sampling means measuring the amplitude of the sound wave. This is done using an analogue to digital converter (ADC).

To convert the analogue data to digital, the sound waves are sampled at regular time intervals. The amplitude of the sound cannot be measured precisely, so approximate values are stored.

The x-axis shows the time intervals when the sound was sampled (1 to 21), and the y-axis shows the amplitude of the sampled sound to 10.

At time interval 1, the approximate amplitude is 10; at time interval 2, the approximate amplitude is 4, and so on for all 20 time intervals. Because the amplitude range in Figure 1.9 is 0 to 10, then 4 binary bits can be used to represent each amplitude value (for example, 9 would be represented by the binary value 1001). Increasing the number of possible values used to represent sound amplitude also increases the accuracy of the sampled sound (for example, using a range of 0 to 127 gives a much more accurate representation of the sound sample than using a range of, for example, 0 to 10). The number of bits per sample is known as the **sampling resolution** (also known as the **bit depth**). So, in our example, the sampling resolution is 4 bits.



Sampling rate is the number of sound samples taken per second. This is measured in hertz (Hz), where 1Hz means 'one sample per second'.

So how is sampling used to record a sound clip?

- the amplitude of the sound wave is first determined at set time intervals (the sampling rate)
- this gives an approximate representation of the sound wave
- each sample of the sound wave is then encoded as a series of binary digits. Using a higher sampling rate or larger resolution will result in a more faithful representation of the original sound source. However, the higher the sampling rate and/or sampling resolution, the greater the file size.

CDs have a 16-bit sampling resolution and a 44.1kHz sample rate – that is 44100 samples every second. This gives high-quality sound reproduction.

Benefits	Drawbacks
larger dynamic range	produces larger file size
better sound quality	takes longer to transmit/download music files
less sound distortion	requires greater processing power

Data Representation

Data Storage and compression

Zain Merchant

Units of data storage

A **binary digit** (or **bit**) is the fundamental unit of data storage, and will have a value of 0 or 1. A group of eight bits is called a **byte**. Four-bit numbers are called a **nibble**.

Historically, storage capacity was expressed using the metric prefixes of **kilo** (1,000), **mega** (1,000,000), etc. Since 1998 there has been a move towards using the special prefixes developed to more accurately represent binary values (as per the International System of Units (SI) definition). For example, a kibibyte is equal to 1,024 bytes, whereas a kilobyte is equal to 1,000 bytes.

The differences between the two systems are shown below, pay close attention to which letters are capitalised or not:

Name	Notation	Power of 10	Value
kilobyte	KB	10^3	1,000 bytes
megabyte	MB	10^6	1,000,000 bytes
gigabyte	GB	10^9	1,000,000,000 bytes
terabyte	TB	10^{12}	1,000,000,000,000 bytes

Name	Notation	Power of 2	Value
kibibyte	KiB	2^{10}	$1024^1 = 1,024$ bytes
mebibyte	MiB	2^{20}	$1024^2 = 1,048,576$ bytes
gibibyte	GiB	2^{30}	$1024^3 = 1,073,741,824$ bytes
tebibyte	TiB	2^{40}	$1024^4 = 109,951,162,776$ bytes

Data storage and file compression

Calculation of file size

The file size of an image is calculated as:

image resolution (in pixels) $\times$ colour depth (in bits)

The size of a mono sound file is calculated as:

sample rate (in Hz) $\times$ sample resolution (in bits) $\times$ length of sample (in seconds)

For a stereo sound file, you would then multiply the result by two.

Worked example

A photograph is 1024×1080 pixels and uses a colour depth of 32 bits. How many photographs of this size would fit onto a memory stick of 64GiB?

1. Multiply number of pixels in vertical and horizontal directions to find total number of pixels = $(1024 \times 1080) = 1\,105\,920$ pixels
2. Now multiply number of pixels by colour depth then divide by 8 to give the number of bytes = $1105920 \times 32 = 35389440/8$ bytes = 4423680 bytes
3. 64 GiB = $64 \times 1024 \times 1024 \times 1024 = 68\,719\,476\,736$ bytes
4. Finally divide the memory stick size by the files size = $68\,719\,476\,736/4\,423\,680 = 15\,534$ photos.

Worked example

A camera detector has an array of 2048 by 2048 pixels and uses a colour depth of 16. Find the size of an image taken by this camera in MiB.

1. Multiply number of pixels in vertical and horizontal directions to find total number of pixels = $(2048 \times 2048) = 4194304$ pixels
2. Now multiply number of pixels by colour depth = $4194304 \times 16 = 67108864$ bits
3. Now divide number of bits by 8 to find the number of bytes in the file = $(67108864)/8 = 8388608$ bytes
4. Now divide by 1024×1024 to convert to MiB = $(8388608)/(1048576) = 8$ MiB.

Worked example

An audio CD has a sample rate of 44100 and a sample resolution of 16bits. The music being sampled uses two channels to allow for stereo recording. Calculate the file size for a 60-minute recording.

1. Size of file = sample rate (in Hz) $\times$ sample resolution (in bits) $\times$ length of sample (in seconds)
2. Size of sample = $(44100 \times 16 \times (60 \times 60)) = 2540160000$ bits
3. Multiply by 2 since there are two channels being used = 5080320000 bits
4. Divide by 8 to find number of bytes = $(5080320000)/8 = 635040000$
5. Divide by 1024×1024 to convert to MiB = $635040000 / 1048576 = 605$ MiB.

Data compression

The calculations previously show that sound and image files can be very large. It is therefore necessary to reduce (or compress) the size of a file for the following reasons:

- to save storage space on devices such as the hard disk drive/solid state drive
- to reduce the time taken to stream a music or video file
- to reduce the time taken to upload, download or transfer a file across a network
- the download/upload process uses up network bandwidth – this is the maximum rate of transfer of data across a network, measured in bits per second. This occurs whenever a file is downloaded, for example, from a server. Compressed files contain fewer bits of data than uncompressed files and therefore use less bandwidth, which results in a faster data transfer rate.
- reduced file size also reduces costs. For example, when using cloud storage, the cost is based on the size of the files stored. Also an internet service provider (ISP) may charge a user based on the amount of data downloaded.

Lossy file compression

With this technique, the file compression algorithm eliminates unnecessary data from the file. This means the original file cannot be reconstructed once it has been compressed.

Lossy file compression results in some loss of detail when compared to the original file.

The algorithms used in the lossy technique have to decide which parts of the file need to be retained and which parts can be discarded.

For example, when applying a lossy file compression algorithm to:

- an image, it may reduce the resolution and/or the bit/colour depth
- a sound file, it may reduce the sampling rate and/or the resolution.

Lossy file compression algorithms

- MPEG-3 (MP3) and MPEG-4 (MP4)
- JPEG.

MP3

MP3 files are used for playing music on computers or mobile phones. This compression technology will reduce the size of a normal music file by about 90%. While MP3 music files can never match the sound quality found on a DVD or CD, the quality is satisfactory for most general purposes.

But how can the original music file be reduced by 90% while still retaining most of the music quality? Essentially the algorithm removes sounds that the human ear can't hear properly. For example:

- removal of sounds outside the human ear range
- if two sounds are played at the same time, only the louder one can be heard by the ear, so the softer sound is eliminated. This is called perceptual music shaping.

JPEG

When a camera takes a photograph, it produces a raw bitmap file which can be very large in size. These files are temporary in nature. JPEG is a lossy file compression algorithm used for bitmap images. As with MP3, once the image is subjected to the JPEG compression algorithm, a new file is formed and the original file can no longer be constructed.

The JPEG file reduction process is based on two key concepts:

- human eyes don't detect differences in colour shades quite as well as they detect differences in image brightness (the eye is less sensitive to colour variations than it is to variations in brightness)
- by separating pixel colour from brightness, images can be split into 8×8 pixel blocks, for example, which then allows certain 'information' to be discarded from the image without causing any real noticeable deterioration in quality.

Lossless file compression

With this technique, all the data from the original uncompressed file can be reconstructed. This is particularly important for files where any loss of data would be disastrous (e.g. when transferring a large and complex spreadsheet or when downloading a large computer application).

Lossless file compression is designed so that none of the original detail from the file is lost.

Run-length encoding (RLE)

- it is a form of lossless/reversible file compression
- it reduces the size of a string of adjacent, identical data (e.g. repeated colours in an image) a repeating string is encoded into two values:
 - the first value represents the number of identical data items (e.g. characters) in the run
 - the second value represents the code of the data item (such as ASCII code if it is a keyboard character)
- RLE is only effective where there is a long run of repeated units/bits.

Using RLE on text data

Consider the following text string: 'aaaaabbbbccddddd'. Assuming each character requires 1byte then this string needs 16bytes. If we assume ASCII code is being used, then the string can be coded as follows:

a	a	a	a	A	b	b	b	b	c	c	d	d	d	d	d
05 97					04 98				02 99		05 100				

This means we have five characters with ASCII code 97, four characters with ASCII code 98, two characters with ASCII code 99 and five characters with ASCII code 100. Assuming each number in the second row requires 1 byte of memory, the RLE code will need 8 bytes. This is half the original file size.

One issue occurs with a string such as 'cdcdcdcdcd' where RLE compression isn't very effective. To cope with this, we use a flag. A flag preceding data indicates that what follows are the number of repeating units (for example, 255 05 97 where 255 is the flag and the other two numbers indicate that there are five items with ASCII code 97). When a flag is not used, the next byte(s) are taken with their face value and a run of 1 (for example, 01 99 means one character with ASCII code 99 follows).

Consider this example:

String	aaaaaaaa	bbbbbbbbbb	c	d	c	d	c	d	eeeeeeee
Code	08 97	10 98	01 99	01 100	01 99	01 100	01 99	01 100	08 101

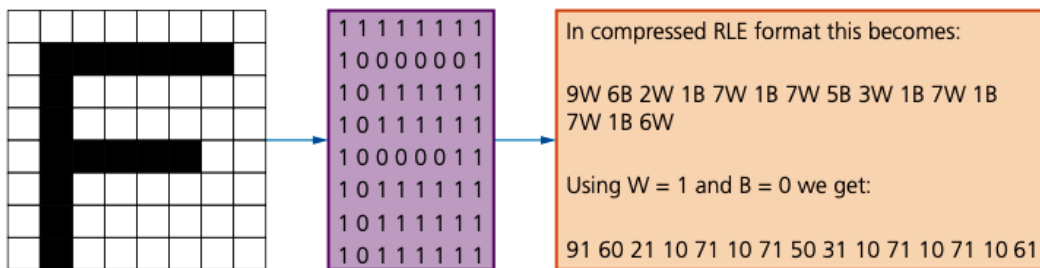
The original string contains 32 characters and would occupy 32 bytes of storage. The coded version contains 18 values and would require 18 bytes of storage. Introducing a flag (255 in this case) produces:

255 08 97		255 10 98		99 100	99 100	99 100		255 08 101
-----------	--	-----------	--	--------	--------	--------	--	------------

Using RLE with images

Worked example

Figure shows the letter 'F' in a grid where each square requires 1 byte of storage. A white square has a value 1 and a black square a value of 0:

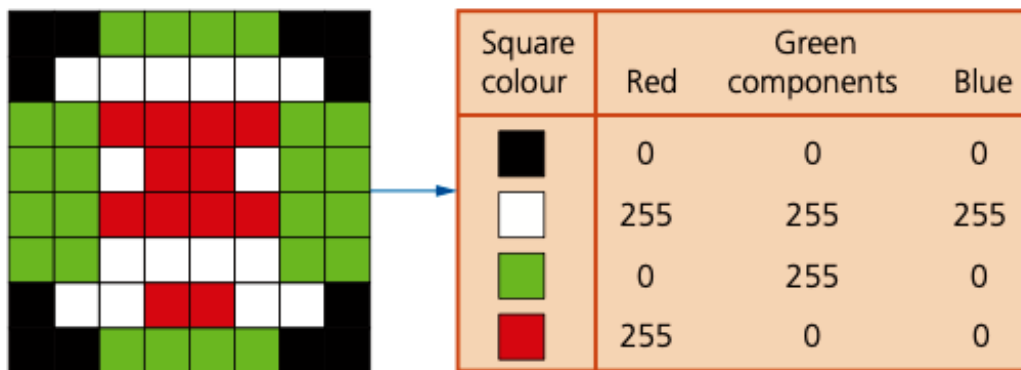


The 8×8 grid would need 64bytes; the compressed RLE format has 30 values, and therefore needs only 30bytes to store the image.

Using RLE with images

Worked example

Figure shows an object in four colours. Each colour is made up of red, green and blue (RGB) according to the code on the right.



This produces the following data: 2 0 0 0 4 0 255 0 3 0 0 0 6 255 255 255 1 0 0 0 2 0 255 0 4 255 0 0 4 0 255 0 1 255 255 255 2 255 0 0 1 255 255 255 4 0 255 0 4 255 0 0 4 0 255 0 4 255 255 255 2 0 255 0 1 0 0 0 2 255 255 255 2 255 0 0 2 255 255 255 3 0 0 0 4 0 255 0 2 0 0 0.

The original image (8×8 square) would need 3bytes per square (to include all three RGB values). Therefore, the uncompressed file for this image is $8 \times 8 \times 3 = 192\text{bytes}$.

The RLE code has 92 values, which means the compressed file will be 92bytes in size. This gives a file reduction of about 52%. It should be noted that the file reductions in reality will not be as large as this due to other data which needs to be stored with the compressed file (e.g. a file header).